Actividad Sesión 2: Análisis de algoritmos y notación Big O

Asignatura: Estructura de Datos I | Programa: Ingeniería de Software

1. Clasificación de fragmentos

Fragmento	Operaciones aproximadas	Big O	Justificación
Acceder a vector[5]	1	O(1)	Accede directamente a una posición; no recorre el arreglo.
Recorrer un arreglo	n	O(n)	El ciclo visita cada elemento una vez.
Dos ciclos anidados	n × n	$O(n^2)$	Por cada iteración externa se ejecutan n iteraciones internas.
Dividir n entre 2 repetidamente	$\log_2(n)$	O(log n)	En cada repetición el valor se reduce a la mitad.
Ciclo n + reducción logarítmica	$n \times \log_2(n)$	O(n log n)	El ciclo externo se ejecuta n veces y el interno aproximadamente $\log_2(n)$ veces.

2. Conteo teórico

Para los logaritmos se utiliza base 2. Los valores son aproximados.

n	O(1)	$O(\log_2 n)$	O(n)	$O(n \log_2 n)$	$O(n^2)$
10	1	3.32	10	33.22	100
100	1	6.64	100	664.39	10,000
1,000	1	9.97	1,000	9,965.78	1,000,000
10,000	1	13.29	10,000	132,877.12	100,000,000

3. Preguntas de análisis

1. ¿Qué diferencia existe entre tiempo de ejecución y complejidad Big O?

El tiempo de ejecución indica cuánto tarda un programa en una ejecución concreta. Big O describe cómo crece el trabajo del algoritmo cuando aumenta el tamaño de entrada.

2. ¿Por qué O(1) se considera una complejidad eficiente?

Porque realiza una cantidad constante de operaciones, independientemente del tamaño de los datos.

3. ¿Qué ocurre con un algoritmo O(n) cuando duplicamos el tamaño de entrada?

Aproximadamente se duplica la cantidad de operaciones, porque el crecimiento es proporcional a n.

4. ¿Qué ocurre aproximadamente con un algoritmo $O(n^2)$ cuando duplicamos n?

La cantidad de operaciones se cuadruplica, porque $(2n)^2 = 4n^2$.

5. ¿Por qué los ciclos anidados suelen relacionarse con $O(n^2)$?

Cuando ambos ciclos recorren n elementos, el ciclo interno se ejecuta n veces por cada iteración del externo: n × n.

6. ¿Qué característica permite reconocer un algoritmo O(log n)?

En cada iteración reduce el tamaño del problema, por ejemplo, dividiéndolo entre dos.

7. ¿Cuál de los dos algoritmos experimentales presentó mayor crecimiento?

El algoritmo cuadrático $O(n^2)$, porque su número de operaciones crece mucho más rápido que el del algoritmo lineal O(n).

8. ¿Los resultados medidos fueron exactamente iguales entre las diferentes ejecuciones? ¿Por qué?

No necesariamente. Los tiempos pueden variar por el procesador, la memoria, otros procesos activos y las optimizaciones de Java.

9. Si tuviera que procesar diez millones de registros, ¿qué tipo de complejidad preferiría?

Preferiría, cuando el problema lo permita, O(1), O(log n) u O(n), porque crecen más lentamente que $O(n^2)$.

10. ¿Por qué un ingeniero de software debe analizar la eficiencia antes de seleccionar un algoritmo?

Porque debe procurar que el programa funcione correctamente y también responda en tiempos razonables, especialmente cuando aumenta la cantidad de datos.

4. Reto adicional

No es $O(n^2)$, sino O(n). Los dos ciclos están uno después del otro, no anidados. Cada uno realiza n iteraciones, por lo que el total es 2n. Al eliminar la constante, la complejidad es O(n).

5. Pregunta orientadora

Elegiría el programa que realiza aproximadamente 1.000 operaciones, porque para la misma cantidad de información requiere menos trabajo que uno que realiza 1.000.000 de operaciones.

6. Conclusión personal

Mediante esta actividad comprendí la importancia de analizar la eficiencia de los algoritmos utilizando la notación Big O. Aprendí que no todos los algoritmos tienen el mismo comportamiento cuando aumenta la cantidad de datos y que algunos pueden tardar mucho más que otros en realizar una misma tarea. También comprendí las diferencias entre las complejidades constantes, logarítmicas, lineales y cuadráticas. Finalmente, considero que como futura ingeniera de software es importante seleccionar algoritmos eficientes que permitan desarrollar programas capaces de procesar grandes cantidades de información y ofrecer un buen rendimiento.